

YOUR 12-WEEK TECHNOLOGY CAREER DISCOVERY PROGRAMME

Hasthi Jayasundera

A handbook for you — everything you need to run this programme yourself, week by week, starting Monday.

12 weeks · ~25 hours a week · one Friday review with your mentor

1. Welcome — What This Is

You've just finished your A/Levels and you've got real free time right now. This programme turns the next 12 weeks into something closer to a structured internship than a study break — you'll work real hours, build real things, and get honest feedback every week, so that by the end you (and your mentor) actually know which direction in technology fits you, instead of guessing.

This isn't just a coding course. Over 12 weeks you'll get a proper, hands-on look at nine different technology directions — software engineering, cybersecurity, networking, cloud, DevOps, data, AI, embedded electronics, and product-building — plus a final week where you choose everything yourself. Along the way you'll build foundational skills (Linux, Python, Git) that matter no matter which direction you end up choosing.

What you're actually being asked to do

- Work roughly 25 hours a week, Monday–Friday, following the daily routine in Section 4
- Build something real every week — not just read or watch tutorials
- Show up to a Friday review with your mentor, ready to explain and defend what you built
- Use AI the right way — Section 3 spells out exactly what that means
- Keep a five-minute daily journal (Section 9) — it's the best record of what actually happened

What a good outcome looks like

There isn't one "correct" ending to this. All of the following count as the programme working:

- You find one direction you're clearly best at and enjoy most — great, that becomes the plan
- Two or three directions stand out — that's still useful; you narrow further from there
- You do fine everywhere but don't really enjoy any of it — that's real information too, worth taking seriously rather than ignoring

It's fine to struggle, get stuck, or have a rough week. It is not fine to fake understanding to avoid that — the whole point is to find out what's actually true, not to look good.

2. How This Works

- Treat this like a job, not a hobby: roughly 25 hours a week, Monday to Friday, during normal working hours.
- Come to the Friday review having actually built something — not "studied about" something.
- Everything you submit, you should be able to explain and defend without notes and without AI open in another tab.
- Use AI within the policy in Section 3, not around it. The checks in that section will be used every single week.
- Keep the daily journal (Section 9). Five minutes. Do it even on the days that felt like nothing happened.
- If you're genuinely stuck, ask your mentor for help. That's completely different from asking AI to produce the answer for you, and it's not a shortcut you should feel bad about using.
- Your mentor is protecting 30–45 minutes every Friday for you, no exceptions — treat that time as seriously as they do.

3. AI Usage Policy

“AI can teach you and accelerate you. AI cannot prove that you understand something.”

This programme is happening in 2026, and learning to use AI tools well — ChatGPT, Claude, Gemini, GitHub Copilot, and others — is a real, valuable skill. This policy isn't about avoiding AI. It's about making sure it speeds up your understanding instead of replacing it.

You can use AI to:

- Explain a concept you don't understand, in a different way than your primary resource did
- Point you toward where a bug likely is (not fix it for you)
- Explain an error message
- Offer an alternative explanation when the first one didn't land
- Give you feedback on something you've already written or built
- Research an unfamiliar concept, tool, or term
- Later in the programme, deliberately learn AI-assisted engineering as its own skill

You should not use AI to:

- Generate code, an assignment, or documentation that you then submit as your own
- Write your journal entries for you
- Write the README or reflection that's supposed to show your own understanding
- Answer viva questions in the moment, or rehearse exact answers to expected questions beforehand and recite them

The standard you're being held to

You should be able to independently explain, modify, and debug anything you submit, with no notes and no AI open, whenever your mentor asks. If you can't do that, the work doesn't count as done yet — no matter how good it looks.

How your mentor will check this

- Looking at your commit history — one giant commit dumped at the end after a long silence usually prompts a question, not an accusation
- Asking about any vocabulary or pattern in your code or writing that's well beyond what's been covered so far — a fair, friendly “walk me through this bit”
- The “change one thing” test — asking you to change a variable name, data type, or small requirement live
- The “break it on purpose” test — introducing a small bug and watching how you debug it
- From Week 10 onward, asking you to point out at least one place AI was wrong, unhelpful, or oversimplified
- Just asking you directly what AI helped with that week — answering honestly is always the right move, even if the honest answer is “AI wrote most of this and I don't fully understand it yet”. That's useful information, not something to be punished for admitting.

4. Your Daily Routine

This targets about 25 hours a week, Monday to Friday. It's built the way a real junior engineer's day works: structured learning, then hands-on practice, then building, then reflection. Keep the four blocks and their order — you document what you actually built, not what you're about to build, so build first.

Time	Block	What you're doing
9:00 – 10:30	Structured learning	Work through that week's primary resource. Take active notes — don't just watch.
10:45 – 12:15	Practical exercises	Small, self-contained exercises that apply what you just learned.
12:15 – 1:30	Break	Lunch, away from the screen.
1:30 – 3:00	Building / project work	Work on that week's main project.
3:00 – 3:30	Journal & documentation	Fill in your daily journal (Section 9) and update your project README.

Friday is a bit different

Time	Block
9:00 – 10:30	Structured learning (finish the week's material)
10:45 – 12:15	Practical exercises / finish the deliverable
1:30 – 2:30	Polish: README, demo recording, journal, repo cleanup
2:30 – 3:15	Your mentor review (Section 8)
3:15 – 3:30	Rate your own week (enjoyment, aptitude — see Section 8)

5. Your Full 12-Week Curriculum

Every week follows the same shape: one primary resource (deliberately just one, not a giant list), the exact topics to cover, the exercises to do, what you need to deliver by Friday, a stretch challenge if you want to push further, the questions your mentor will ask you, the live challenge to expect, and what evidence you need to have ready.

Week 1 — Computer Systems Fundamentals & Linux Basics

Module 1: Computer Systems + Linux · Target: 25 hrs

Primary resource	Linux Journey (linuxjourney.com) — free, self-paced, no signup required. Complete the "Grasshopper" (Linux basics) and start "Camel" (users/permissions) tracks.
Backup resource	freeCodeCamp "Linux for Beginners" video course (YouTube, free) as an alternative explanation style if a topic doesn't click.
Topics to cover	• How a computer actually works: CPU, RAM, storage, the role of an operating system • Installing VirtualBox (or VMware Player) and an Ubuntu Desktop/Server VM • The Linux filesystem hierarchy (/etc, /home, /var, /usr, etc.) • Navigating entirely by terminal: cd, ls, pwd, mkdir, mv, cp, rm, find, grep • Users, groups, and permissions (chmod, chown, the rwx model) • Package management basics with apt
Exercises to do	• Install the VM from scratch, screenshot every stage, log any error you hit and how you solved it • Spend one full working session doing everything through the terminal only (no file-manager GUI) • Create 2 extra users and a group; configure permissions so only that group can access a folder • Break a permission on purpose, then fix it, and write down what you did in both directions
What you need to deliver by Friday	GitHub repo "week01-linux-fundamentals" containing a markdown log of every command used (with a one-line note on what it does), a screen recording of the VM setup, and the users/permissions exercise documented.
Stretch challenge (optional)	Attempt OverTheWire "Bandit" wargame, levels 0–5 (free, browser + SSH), and log your approach even for levels you don't finish.
Questions your mentor will ask	• Explain what happens, step by step, when you type `ls -la` and press enter. • What's the difference between a process and a program? • Show me a real scenario where the wrong permission would break something — then break it and fix it live.
Live challenge to expect	Your mentor will deliberately misconfigure a permission or ownership setting on a file in your VM. You need to diagnose the symptom and fix it live, narrating your reasoning out loud — no searching, no AI.
Evidence to have ready	• Screen/terminal recording

	• Command log in the repo • Completed permissions exercise • Daily journal entries
Why this week matters	This week is about whether you can navigate a system with no GUI to lean on and still think clearly. It's the first data point on general systems aptitude, and an early hint toward SysAdmin / Cloud / Cybersecurity if it clicks for you.

Week 2 — Linux Deep Dive, Shell Scripting & SSH

Module 1: Computer Systems + Linux · Target: 25 hrs

Primary resource	MIT "The Missing Semester of Your CS Education" (missing.csail.mit.edu — free, lecture notes + videos). Cover the Shell Tools, Editors, and Debugging & Profiling lectures.
Backup resource	Remaining Linux Journey tracks (Mountain, Volcano) for anything the MIT material assumes as background.
Topics to cover	- Processes, environment variables, and PATH - Bash scripting basics: variables, loops, conditionals, arguments - Package management in more depth; installing/removing/troubleshooting a package - SSH: generating a key pair, key-based auth, connecting to a remote machine - A basic troubleshooting method: reproduce → isolate → read the error → fix → verify
Exercises to do	- Write 3 small bash scripts: a folder backup script, a log-file keyword searcher, a disk-usage reporter - Generate an SSH key pair and connect to a free remote VM (e.g. a free-tier cloud instance, or a second local VM) using key auth only - Have your mentor (or a script) silently break one of your own scripts; find and fix the bug
What you need to deliver by Friday	GitHub repo "week02-linux-advanced" with the three scripts, a README explaining what each does and why it's written that way, and a recorded SSH session.
Stretch challenge (optional)	OverTheWire "Bandit" levels 6–15.
Questions your mentor will ask	- Walk me through your backup script, line by line — why this approach and not another? - What is an environment variable, and why does PATH specifically matter? - You SSH into a machine and a command isn't found. Talk me through how you'd figure out why.
Live challenge to expect	Your mentor will introduce a bug into one of your own scripts while you're not looking. You'll debug it live, out loud, with no AI assistance.
Evidence to have ready	- Three working scripts + README - SSH session recording - Bug-fix explained in the journal
Why this week matters	If you enjoy the troubleshooting and automation side of this week more than anything else so far, that's a real signal toward SysAdmin / DevOps / Platform Engineering.

Week 3 — Python Fundamentals I

Module 2: Programming with Python · Target: 25 hrs

Primary resource	Python Essentials 1 — Cisco Networking Academy × Python Institute/OpenEDG (free, self-paced, ~30 hours, certificate available via netacad.com).
Backup resource	The official Python tutorial (docs.python.org/3/tutorial) for a denser, reference-style explanation of anything unclear.
Topics to cover	• Variables and data types • Conditionals (if/elif/else) • Loops (for, while) • Functions, parameters, return values • Core collections: list, tuple, dict, set • String handling and formatting • Basic file I/O • An introduction to exceptions
Exercises to do	• Daily: solve 2–3 small, self-contained problems from scratch (not tutorial-along-coding) — e.g. a palindrome checker, a word-frequency counter, a simple number-guessing game • No copying tutorial code verbatim: type every line yourself and be able to explain it without looking
What you need to deliver by Friday	GitHub repo "week03-expense-tracker-v1": a command-line Expense Tracker that can add, list, and delete expenses, and saves them to a text or CSV file between runs.
Stretch challenge (optional)	Add expense categories and a command that prints a monthly summary.
Questions your mentor will ask	• Why did you choose a dictionary here instead of a list? • What happens in your program if the data file doesn't exist yet? Show me.
Live challenge to expect	Your mentor will ask you to add a specific new feature to the tracker on the spot — for example, a delete-by-index command. You build it live, thinking out loud, no AI.
Evidence to have ready	• Repo with commit history • Recorded live-coding session for the challenge • Journal
Why this week matters	This is the first real test of raw programming aptitude — how comfortable you are with logic and structure. It's the starting point for a Software Engineering / CS signal.

Week 4 — Python Fundamentals II — OOP, Files, APIs & Debugging

Module 2: Programming with Python · Target: 25 hrs

Primary resource	"Automate the Boring Stuff with Python" by Al Sweigart — free to read online (automatetheboringstuff.com). Focus on the OOP, file-handling, and web/API chapters.
Backup resource	Python Essentials 2 (Cisco NetAcad × OpenEDG, free) for a more formal treatment of OOP and exception handling.
Topics to cover	• Basic OOP: classes, objects, methods, attributes • Modules and packages, importing your own code • Reading/writing CSV and JSON • Calling a public API with the `requests` library • Structured exception handling (try/except/finally) • Debugging with print statements and the built-in debugger (pdb)
Exercises to do	• Build a File Organiser: a script that sorts files in a messy folder by type/extension into subfolders • Build a class-based Inventory System (add/update/remove/search items, persist to a file) • Call one free public API (e.g. a weather or currency-exchange API) and handle a failed request gracefully
What you need to deliver by Friday	GitHub repo "week04-python-projects" with both projects, each with its own README explaining the design choices (not just what the code does).
Stretch challenge (optional)	Add a small automated test suite using Python's built-in `unittest` module.
Questions your mentor will ask	• Explain your class design — why these attributes and methods, and not others? • What happens if the API call fails or times out? Show me the code path, then make it fail on purpose.
Live challenge to expect	Your mentor will rename or remove a method your code depends on, or feed your program bad input. You find and fix the failure live.
Evidence to have ready	• Two working repos • Recorded debugging walkthrough • Journal
Why this week matters	Whether you'd rather build something reusable and well-structured than a quick one-off script is an early signal about whether Software Engineering (versus scripting/ops) is the better fit for you.

Week 5 — Git, GitHub & Engineering Workflow

Module 3: Engineering Workflow · Target: 20 hrs

Primary resource	GitHub Skills (skills.github.com) — free, interactive, in-browser courses: "Introduction to GitHub", "Communicate using Markdown", and "Resolving merge conflicts".
Backup resource	Atlassian's free Git tutorials (atlassian.com/git) for clearer conceptual diagrams of branching and merging.
Topics to cover	• git init/add/commit/log; the staging area explained properly • Branching and merging • Deliberately causing and resolving a merge conflict • .gitignore and repo hygiene • GitHub issues and pull requests • Writing meaningful commit messages • Structuring a good README
Exercises to do	• Retrofit proper, meaningful commit history onto the Week 3–4 projects if they weren't already version-controlled that way • Create a second local branch, make conflicting edits on both, merge them, and resolve the conflict by hand • Open at least 3 issues on your own repo describing real improvements, then close them via commits/PRs
What you need to deliver by Friday	All prior project repos brought up to a proper standard (clear commit history + README), plus a documented merge-conflict resolution (before/after + explanation).
Stretch challenge (optional)	Add a minimal GitHub Actions workflow that automatically runs one of your Python scripts on every push.
Questions your mentor will ask	• Walk me through your commit history — why did you split the work into these commits? • Show me the merge conflict you resolved and explain, line by line, what Git couldn't figure out on its own.
Live challenge to expect	You and your mentor will create a fresh conflicting change together in real time. You resolve the conflict live, narrating your reasoning.
Evidence to have ready	• Commit history quality • Merge-conflict screenshots/log • README quality across all repos so far
Why this week matters	This week isn't really about Git — it's about whether you care about process and traceability. That trait matters in every one of the paths this programme is testing.

Week 6 — Networking Fundamentals

Module 4: Networking · Target: 25 hrs

Primary resource	Cisco Networking Academy / Skills for All — "Introduction to Networks" (free, self-paced, includes Cisco Packet Tracer).
Backup resource	Professor Messer's free Network+ video series (professormesser.com) for an alternative explanation of subnetting.
Topics to cover	• OSI and TCP/IP models • IPv4 addressing and subnetting • MAC addresses and ARP • DHCP and DNS • TCP vs UDP • HTTP/HTTPS and ports • Routing and switching basics • NAT and firewalls • Wi-Fi fundamentals
Exercises to do	• Work through subnetting problem sets until he can subnet correctly without a calculator • Build a small topology in Cisco Packet Tracer (PCs, a switch, a router, internet access) • Capture and inspect his own local traffic with Wireshark to see DNS/TCP/HTTP happening for real
What you need to deliver by Friday	Packet Tracer file plus a written design for: "Design the network for a small 20-person company with wired PCs, Wi-Fi, printers, servers and internet access", including a topology diagram, IP addressing scheme, device list, and justification for each choice.
Stretch challenge (optional)	Add a separate guest Wi-Fi VLAN and a basic access-control rule to the design.
Questions your mentor will ask	• Why did you subnet it this way rather than another way? • Walk me through exactly what happens on the network from the moment your laptop requests a webpage to the page appearing — DNS, TCP handshake, HTTP — with no notes.
Live challenge to expect	Your mentor will change the requirement live — for example, "now it's 50 people and a server room" — and you'll redesign the topology and addressing on the spot, on paper or a whiteboard, and defend the new design.
Evidence to have ready	• Packet Tracer file • Design document • Recorded explanation of the redesign
Why this week matters	Doing well here and actually enjoying it is one of the clearest early signals toward Networking, Cybersecurity, or Cloud/Infrastructure.

Week 7 — Cybersecurity Fundamentals

Module 5: Cybersecurity · Target: 25 hrs

Primary resource	TryHackMe — free rooms in the "Pre Security" and "Introduction to Cyber Security" learning paths (legal, sandboxed training environment).
Backup resource	Cisco Skills for All "Introduction to Cybersecurity" course (free) for a more conceptual, exam-style treatment of the same ground.
Topics to cover	• The CIA triad and core security concepts • Common threats and vulnerabilities • Linux security basics • Network security basics • Reconnaissance concepts and Nmap fundamentals • Web security fundamentals (OWASP Top 10, at introduction level) • Authentication and permissions • Basic defensive security • Ethical and legal boundaries, including Sri Lanka's Computer Crime Act
Exercises to do	• Complete the free rooms in the TryHackMe Pre Security path • Run Nmap against his own local VM only — never against a system he doesn't own or have written permission to test • Configure basic firewall rules (ufw) on his Linux VM and verify they work
What you need to deliver by Friday	A written, own-words summary of 3 completed TryHackMe rooms: what the room taught, what he actually did, and what he'd do differently. Copy-pasted room text is treated as a compliance failure (see AI & Integrity Policy).
Stretch challenge (optional)	Attempt one "easy"-rated CTF-style room and document the approach honestly, even if he doesn't finish it.
Questions your mentor will ask	• Explain, in your own words, what Nmap is actually doing on the network when you run a scan. • Why is it illegal and unethical to do this against a system you don't own? Where exactly is the line?
Live challenge to expect	Your mentor will pick one specific finding from your own scan or exercise and ask you to interpret it live, unaided.
Evidence to have ready	• TryHackMe completion screenshots • Own-words write-ups • Journal entry explicitly discussing the legal/ethical boundary
Why this week matters	Curiosity about how systems break or can be broken, and comfort with ambiguous, adversarial-thinking problems, is the clearest Cybersecurity signal in the whole programme.

Week 8 — Cloud & Infrastructure

Module 6: Cloud & Infrastructure · Target: 25 hrs

Primary resource	Microsoft Learn — "Azure Fundamentals" (AZ-900) learning path (free, no credit card required to learn the content).
Backup resource	AWS Skill Builder (skillbuilder.aws) free digital courses. Note: AWS Educate's free credit programme has shrunk since 2023 and now requires registration for a small, time-limited credit — treat Skill Builder's free courses as the more dependable no-cost option, and re-check current terms before relying on AWS credits.
Topics to cover	• What cloud computing actually is (and isn't) • Compute, storage, networking, and databases in the cloud • IAM basics (identities, roles, least privilege) • Virtual machines and basic deployment • DNS in a cloud context • Basic architecture patterns • Infrastructure security basics • Cost awareness
Exercises to do	• Deploy an earlier project (turn the Expense Tracker or File Organiser into a very small web app) to a genuinely card-free free tier — e.g. PythonAnywhere, Render, or Railway's free tier — since many "free tier" cloud offers still require a card • Use Azure/AWS learning content purely to understand the concepts, even if the actual deployment happens on a simpler free host
What you need to deliver by Friday	A live URL where the app is reachable, plus a diagram (hand-drawn photo or a free tool like draw.io) explaining exactly what happens from the moment a user enters the URL to the response appearing.
Stretch challenge (optional)	Add basic logging/monitoring, or a custom subdomain.
Questions your mentor will ask	• Trace a single request through your deployed app, from browser to server and back, in full detail. • What would break first if this suddenly had 10,000 users, and what would you change?
Live challenge to expect	Your mentor will deliberately break your deployed app (or ask you to introduce a specific error) and you'll diagnose it using logs, live.
Evidence to have ready	• Live URL • Architecture diagram • Deployment logs/screenshots
Why this week matters	If you enjoy infrastructure, deployment, and troubleshooting under a bit of pressure, that points toward Cloud Engineering / DevOps / SRE.

Week 9 — Software / Product Engineering

Module 7: Software / Product Engineering · Target: 25 hrs

Primary resource	The Odin Project (theodinproject.com) — free, project-based full-stack curriculum. Use the relevant Python/Flask-adjacent or general web-fundamentals sections; adapt to Python + Flask to build on prior weeks.
Backup resource	freeCodeCamp's Scientific Computing / backend curriculum (free) for additional explanation of any backend concept.
Topics to cover	• Building a small but complete application end-to-end • Basic authentication • Persisting data in a real database (SQLite is enough) • Designing a schema for transactions/categories • Basic reporting/aggregation • A simple, responsive UI (plain HTML/CSS is enough — do not over-focus on frameworks)
Exercises to do	• Build "Family Expense Manager": authentication, transactions, categories, a SQLite database, monthly totals, basic reporting, and a usable, responsive UI • Deploy it using the Week 8 hosting setup
What you need to deliver by Friday	Working GitHub repo + live deployed URL + README + a self-recorded 3–5 minute demo walkthrough.
Stretch challenge (optional)	Add CSV export of transactions, or a simple chart of monthly spending.
Questions your mentor will ask	• Why this database schema and not another one? • Show me the code path for adding a transaction, end to end, and explain every step.
Live challenge to expect	Your mentor will request a live schema change — for example, "add a notes field to each transaction" — that you implement end-to-end (database, backend, UI) without AI assistance.
Evidence to have ready	• Repo + live URL • Demo video • Live schema-change session
Why this week matters	Genuinely enjoying end-to-end ownership — building something a real person would actually use — is the clearest Software Engineering / Product signal in the programme.

Week 10 — Data Analysis & Responsible AI Use

Module 8: Data + AI · Target: 25 hrs

Primary resource	Kaggle Learn (kaggle.com/learn) — free micro-courses: Python, Pandas, Data Visualization, and Intro to Machine Learning.
Backup resource	freeCodeCamp's "Data Analysis with Python" certification course (free) for a slower-paced alternative.
Topics to cover	• Data structures for data work • Reading and cleaning CSV/JSON data with pandas • Basic statistics (mean, median, distributions, outliers) • Visualization with matplotlib • Drawing (and stating) conclusions from data honestly • The actual difference between AI, ML, and LLMs • Responsible use of LLM APIs — what they're good and bad at
Exercises to do	• Choose a real dataset (Kaggle or an open Sri Lankan dataset) relevant to something he's curious about • Clean it, analyse it, visualise at least two findings, and write the conclusions in plain English • Call one free-tier LLM API to summarise the dataset's findings, then critique what the AI got wrong, oversimplified, or hallucinated
What you need to deliver by Friday	A notebook (Jupyter/Colab or a plain .py + saved plots) in the repo, plus a one-page written findings summary and a documented AI critique.
Stretch challenge (optional)	Repeat the analysis on a second, messier dataset without step-by-step guidance.
Questions your mentor will ask	• What does this chart actually tell us — and just as important, what does it NOT tell us? • In your own words: what's the actual difference between AI, ML, and an LLM? • Where exactly was the AI wrong or misleading when you used it this week?
Live challenge to expect	Your mentor will hand you a different, messier dataset on the spot; you clean and analyse a subset of it live.
Evidence to have ready	• Notebook • Findings summary • Documented AI critique
Why this week matters	Enjoying quantitative reasoning and pattern-finding, especially while staying honest about uncertainty, points toward Data Engineering / Data Science / AI Engineering.

Week 11 — Electronics, Embedded Systems & IoT

Module 9: Electronics / Embedded / IoT · Target: 25 hrs

Primary resource	Random Nerd Tutorials (randomnerdtutorials.com) — free, high-quality, current ESP32/Arduino guides. Start with "Getting Started with ESP32" and the DHT22/DS18B20 temperature sensor guides.
Backup resource	Official Arduino and Espressif (ESP32) documentation for authoritative reference when a tutorial is unclear.
Topics to cover	• Voltage/current basics, just enough to wire safely • Digital vs. analog signals • GPIO • Reading a sensor (e.g. DHT22 or DS18B20 temperature sensor) • Microcontrollers and embedded C/C++ via the Arduino IDE • Serial communication • Wi-Fi connectivity on the ESP32 • Basic IoT concepts: device → network → viewer
Exercises to do	• Wire and blink an LED; understand the resistor's role • Wire and read a temperature sensor, printing values over serial • Extend the project to Wi-Fi so a reading can be viewed from another device (a simple on-device web page, or publishing to a free service such as a basic MQTT broker or ThingSpeak's free tier)
What you need to deliver by Friday	Given only the problem statement — "build a device that measures temperature and lets the reading be viewed from another device" — a working device plus its code repo plus photos/video of the physical build and a recorded live demo. He should research how to achieve this himself; do not hand him a complete tutorial for the whole build.
Stretch challenge (optional)	Add a second sensor (e.g. humidity) or an alert/LED that triggers past a threshold.
Questions your mentor will ask	• Walk me through the circuit — why this resistor value, why this pin? • What happens if the Wi-Fi connection drops mid-run — does your code handle it, and how do you know?
Live challenge to expect	Your mentor will change the requirement live — "now also show humidity" or "alert if temperature exceeds 30°C" — and you'll modify the hardware and/or code on the spot.
Evidence to have ready	• Photos/video of the physical build • Code repo • Recorded live demo of a working reading
Why this week matters	This is the week most likely to reveal whether physical, hands-on hardware excites you more than pure software — especially worth watching given your Physics grade.

Week 12 — Capstone — Free Choice

Capstone · Target: 25 hrs (or more, voluntarily)

Primary resource	None assigned. He chooses the problem and the technology himself. This is intentional — the choice itself is part of the assessment.
Backup resource	—
Topics to cover	• Whatever the chosen project requires him to learn
Exercises to do	• The brief he receives is exactly this: "You have one week. Build something useful or interesting. You choose the problem and the technology." • Do not steer him toward a particular technology or domain, even if he asks. If he asks for direction, respond with a question, not an answer.
What you need to deliver by Friday	Whatever he chooses to build, finished to whatever state he reaches by Friday, plus a final demo and technical viva.
Stretch challenge (optional)	Continuing past the minimum requirement voluntarily is itself a tracked signal, not a bonus task to assign.
Questions your mentor will ask	• Why this problem, and why this technology? • What did you look up or learn that nobody told you to learn? • What would you do differently if you had another week?
Live challenge to expect	A full technical viva plus a live modification request specific to whatever you built — decided in the moment.
Evidence to have ready	• Repo (or physical build + documentation) • Final demo video • Viva notes
Why this week matters	This week tests everything at once: the problem you chose, how ambitious you were, what you researched without being told to, and whether setbacks stopped you or not. It carries more weight than any single earlier week.

6. Deliverables – Quick Reference

A one-line reminder of what needs to exist by each Friday. Full detail is in Section 5.

Wk	Deliverable
1	GitHub repo "week01-linux-fundamentals" containing a markdown log of every command used (with a one-line note on what it does), a screen rec...
2	GitHub repo "week02-linux-advanced" with the three scripts, a README explaining what each does and why it's written that way, and a recorded...
3	GitHub repo "week03-expense-tracker-v1": a command-line Expense Tracker that can add, list, and delete expenses, and saves them to a text or...
4	GitHub repo "week04-python-projects" with both projects, each with its own README explaining the design choices (not just what the code does...
5	All prior project repos brought up to a proper standard (clear commit history + README), plus a documented merge-conflict resolution (before...
6	Packet Tracer file plus a written design for: "Design the network for a small 20-person company with wired PCs, Wi-Fi, printers, servers and...
7	A written, own-words summary of 3 completed TryHackMe rooms: what the room taught, what he actually did, and what he'd do differently. Copy-...
8	A live URL where the app is reachable, plus a diagram (hand-drawn photo or a free tool like draw.io) explaining exactly what happens from th...
9	Working GitHub repo + live deployed URL + README + a self-recorded 3–5 minute demo walkthrough.
10	A notebook (Jupyter/Colab or a plain .py + saved plots) in the repo, plus a one-page written findings summary and a documented AI critique.
11	Given only the problem statement — "build a device that measures temperature and lets the reading be viewed from another device" — a working...
12	Whatever he chooses to build, finished to whatever state he reaches by Friday, plus a final demo and technical viva.

7. GitHub & Evidence Standard

From Week 5 onward, this is what “done” means for every project:

- ☐ A GitHub repository (public, or shared with your mentor)
- ☐ A commit history with multiple, meaningfully-spaced commits and honest messages — not one dump at the end
- ☐ A README in your own words: what it does, why you built it this way, how to run it, what you'd improve
- ☐ A 3–5 minute recorded demo, spoken live — not read from a script
- ☐ A completed mentor review (Section 8)
- ☐ A completed live modification or debugging challenge
- ☐ A journal entry for every day you worked on it (Section 9)

Naming your repos

weekNN-short-project-name — for example week03-expense-tracker-v1, week09-family-expense-manager. Keeping this consistent means your whole 12 weeks of work is easy to look back on at the end.

8. What to Expect in Your Friday Review

Every Friday, 30–45 minutes with your mentor. The shape is the same every week — only the specific questions and challenge change (see that week's entry in Section 5). Knowing the shape in advance isn't cheating; the point is for you to genuinely be ready, not to be caught off guard.

1. Show your mentor what you built.
2. Explain how it works, without looking at documentation or notes.
3. Talk about the hardest problem you hit this week.
4. Show your Git history.
5. Explain one technical concept from the week, from scratch, as if you were teaching it.
6. A live modification challenge (that week's specific version — Section 5).
7. A live debugging challenge (that week's specific version — Section 5).
8. What AI helped you with this week.
9. One example where AI was wrong or unhelpful this week.
10. Something you learned without being explicitly asked to.
11. Rate how much you enjoyed this week's field, 1–5, and say why.
12. Say honestly whether you'd keep learning this if the programme ended today.

That last handful of questions matter as much as the technical ones. Your honest enjoyment rating — separate from how well you performed — is exactly the data this whole programme exists to collect. A field you're good at but dread is just as important a result as one you're good at and love.

9. Your Daily Journal Template

Five minutes, every day, at the end of the 3:00–3:30 block. Keep it short enough that you'll actually do it. Paste this into a markdown file in that week's repo, or a simple notes app.

Field	Your entry
Date	
Hours worked	
What I learned	
What I built / practised	
Biggest problem	
How I tried solving it	
Did I use AI? If yes, how?	
What did I understand after using AI?	
What still confuses me	
What I want to try tomorrow	

10. Week 12 — Your Capstone

Week 12 is the most important week of the whole programme, because for the first time nobody is choosing the problem or the technology for you. The brief you'll get is exactly this:

“You have one week. Build something useful or interesting. You choose the problem and the technology.”

If you're tempted to ask your mentor for direction, try answering your own question first: what problem are you actually curious about? The choice itself is part of what's being evaluated — not in a judgmental way, but because it's the clearest window into what genuinely interests you when nobody's steering.

What matters this week

- The problem you choose, and why
- Whether you reach for something from the last 11 weeks, or go somewhere new
- What you look up or learn without being told to
- How ambitious the idea is for one week
- How you handle setbacks — push through, simplify, or something else
- Whether you keep working on it after the week is technically over
- Whether you can clearly explain your own decisions afterward

Your final demo and viva

Treat this like the closing interview for the whole programme:

1. Full demo of whatever you built, in whatever state it's in.
2. A complete, unaided walkthrough of how it works.
3. Why this problem — what made it interesting to you?
4. Why this technology — the right tool, or just the comfortable one?
5. A live modification request specific to what you built.
6. “What would you do with another week?”
7. “Looking back across all 12 weeks, what did you enjoy most, and what did you dread?”

11. What Your Results Might Point Toward

This is just for your own context — your mentor will do the full read of your scores at the end, but it helps to know what the programme is actually looking for as you go.

Direction	What tends to point this way
Cybersecurity	Strong Linux, networking, and security weeks; real curiosity about how systems fail or get broken into; you enjoy investigation more than building (Weeks 1, 2, 6, 7 especially).
Cloud / DevOps / SRE	Strong Linux, networking, and cloud weeks; you enjoy automation, deployment, and troubleshooting infrastructure under pressure (Weeks 1, 2, 6, 8).
Software Engineering / CS	Strong programming and app-development weeks; you add features nobody asked for; you enjoy structure and problem-solving (Weeks 3, 4, 5, 9).
Computer Engineering / Embedded	Physics/hardware interest plus programming ability; Week 11 (ESP32/IoT) stands out clearly.
Mechatronics / Automation	Strong pull toward physical systems and machines, not just code — often shows up in how you talk about Week 11, not just the score.
Data Engineering	Strong programming plus real data aptitude; you enjoy cleaning and transforming datasets more than building user-facing features (Week 10).
AI / Machine Learning	Programming plus data plus genuine curiosity about the maths underneath, and willingness to build that maths up further (Week 10).
Product / Tech Entrepreneurship	Strong building ability combined with unusually good problem selection and initiative — this shows up most in Week 12, where you chose everything yourself.

12. What Happens After the 12 Weeks

Nothing about your future education gets decided before Week 12 finishes — that's deliberate. Once your results are in, you and your mentor will look at what actually stood out (both how well you did and how much you enjoyed it) and match that against, education options in Sri Lanka — things like the Open University of Sri Lanka, SLIATE's HNDIT, NVQ-based technical pathways, university external degree programmes, and vendor certifications alongside a degree or diploma.

Almost everything in this handbook is free to do. The one thing worth budgeting for during the programme itself is a small electronics kit for Week 11 (an ESP32 board, a breadboard, a couple of sensors) — inexpensive and easy to source locally in Sri Lanka.

If it turns out a technology career isn't actually the right direction for you, that's a completely fine outcome. The whole point of spending 12 weeks finding this out now, before any money is spent on formal education, is so a mismatch doesn't get locked in just because it seemed like the obvious path.

13. Monday Morning — Your Quick Start

You should be able to read this page on Sunday and start on Monday without anyone explaining anything else to you first.

Time	Action
9:00	Create a professional Gmail account if you don't already have one suitable for this (firstname.lastname style, not a childhood nickname account).
9:15	Create a GitHub account with a professional username.
9:30	Create a top-level GitHub repository named “tech-career-discovery-2026” with a weekNN/ folder for each week ahead.
10:00	Install VirtualBox (or VMware Player) and download an Ubuntu Desktop ISO.
10:30	Start the Ubuntu VM installation — it can run in the background while you read.
10:45	Open linuxjourney.com and start the “Grasshopper” track — this is your actual Week 1 structured-learning block, starting now.
12:15	Break.
1:30	Once the VM install is done, start the Week 1 practical exercises (Section 5, Week 1).
3:00	Write your first daily journal entry (Section 9) — even a short one counts. The habit matters more than the length on day one.
3:20	Read Section 3 (AI Usage Policy) and Section 2 (How This Works) properly, once, all the way through.
3:30	Message your mentor: one line confirming the VM is installed and Week 1 has started.

Good luck. Build things, break things, and be honest about both.